LabelLens

Personal Food Intelligence • 24-Hour Hackathon Build Plan

Scan the label. Understand the evidence. Track what you consume.

BASELINE
Photo → OCR/vision → structured label data → preference matching → evidence + uncertainty

OUR DIFFERENTIATOR
Evidence Lens • Uncertainty Radar • Personal Compatibility Map • Nutrition Memory • Before-You-Consume preview

24-HOUR MVP
Build the complete label-analysis and confirmed-consumption workflow first. Defer live wearables and medical recommendations.

Implementation plan • Architecture • Feature scope • Team roles • 24-hour timeline • Testing and demo

Plan at a glance

This document combines the problem statement's baseline requirements with the proposed longer-term vision. Use the MVP column as the build contract for the 24-hour hackathon; treat wearables and advanced health features as roadmap items.

Section	What it covers
1. Executive summary	
2. What problem are we solving?	
3. Product principles and differentiators	
4. Full user journey	
5. Required baseline solution — compliance checklist	
6. System architecture	
7. Decision logic and status model	
8. Nutrition calculations and consumption ledger	
9. Feature scope: 24-hour MVP vs future roadmap	
10. Recommended stack and repository	
11. Six-person team division	
12. 24-hour execution plan	
13. Acceptance tests and demo script	
14. Safety, privacy and honest limitations	
15. Final deliverables and success criteria	

1. Executive summary

LabelLens is an AI-powered food-label intelligence system that reads packaged-food labels, structures ingredient and nutrition information, compares that evidence with a user's dietary profile, and tracks confirmed consumption over time. It goes beyond a chatbot by returning a traceable compatibility report, showing the exact label evidence behind each finding, and explicitly marking missing, unreadable, ambiguous, or incomplete information as unknown.

Core product loop: Personal profile → Scan label → Extract and validate → Check compatibility → Preview impact → Confirm consumption → Update daily ledger → Detect patterns.

2. What problem are we solving?

Packaged-food labels can be difficult to interpret quickly. Consumers may need to identify allergens, understand ingredients, compare nutrition values with personal preferences, and remember how a product contributes to their daily intake. LabelLens turns the visible label into structured, evidence-backed information and makes the result personal.

- People may miss an ingredient or allergen hidden in a long list.
- Nutrition values may be shown per 100 g/ml, per serving, or per package and are easy to compare incorrectly.
- A product scan is not proof that the user consumed it; tracking needs an explicit confirmation and portion size.
- A missing or unreadable ingredient must not be treated as absent.
- Users need to understand why a flag was raised—not just receive a generic AI answer.

3. Product principles and differentiators

Feature	What it does	Why it matters
Evidence Lens	Links each finding to exact extracted label text or a nutrition field; highlights source text where possible.	Makes results verifiable rather than chatbot-like.
Uncertainty Radar	Separates detected, not found in a complete readable list, missing, unreadable, ambiguous, and incomplete.	Prevents false reassurance.
Personal Compatibility Map	Shows allergy conflicts, dietary checks, nutrition preferences, and unknowns as separate categories.	Personalized without hiding a serious conflict behind one overall score.
Nutrition Memory	Stores only confirmed consumption events with portion, time, and nutrient quantities.	Enables cumulative daily totals and history.
Before-You-Consume preview	Projects the impact of a selected portion before logging it.	Helps users make informed choices before consuming.
Pattern insights	Flags repeated recorded patterns, such as an energy drink logged several days in a row.	Adds longitudinal value without claiming to diagnose disease.

4. Full user journey

A. Create profile

Set allergens, dietary preferences, custom ingredients to avoid, and user-defined nutrition targets. Height, weight, and health context are optional and should not be required for label interpretation.

B. Upload label

Accept a photograph of the ingredient list and/or nutrition table; show a preview and allow a retake.

C. Extract

Use an authorized vision/OCR API and an LLM or equivalent structured-output component to extract ingredients, nutrition facts, units, serving basis, allergen statements, and source text.

D. Validate evidence

Normalize fields, preserve original text, validate units and numeric values, and identify missing, unreadable, ambiguous, cropped, or incomplete information.

E. Compare

Apply deterministic rules for explicit ingredient/allergen matches and numeric thresholds. Use LLM interpretation only where it adds value, with uncertain interpretations left unresolved.

F. Review compatibility report

Show the original image, extracted ingredients, nutrition table, preferences, findings, source evidence, and uncertainty messages.

G. Preview portion impact

Ask how much the user intends to consume; calculate the additional nutrient quantity and projected daily totals.

H. Confirm consumption

Only add a ledger event after the user confirms consumption. A scan or comparison alone must not change intake totals.

I. Track and learn

Display today's totals and history; identify repeated logged patterns and send opt-in reminders when useful.

5. Required baseline solution — compliance checklist

Problem-statement requirement	LabelLens implementation	Acceptance test
Accept a photograph of a label	Upload ingredient and/or nutrition label image; preview and retake.	A JPG/PNG image can be submitted and displayed.
Authorized OCR/vision API + LLM/equivalent structure	Backend calls an authorized image-capable API and validates structured JSON.	A real label produces structured fields, not only free-form prose.
Display extracted ingredients and nutrition; retain source text	Ingredient list, normalized nutrition table, original OCR/source snippets, units and serving basis.	A user can inspect the extracted value and its supporting text.
Preferences/restrictions compared using explicit rules and interpretation	Python matching engine: allergen/custom-ingredient rules, dietary checks, and configurable nutrition targets.	A selected restriction produces a reproducible result from the same evidence.
Evidence highlighted for each flag	Each finding stores source_text, source_type, and optionally image coordinates.	Every conflict or preference flag shows its evidence or explains why evidence is unavailable.
Absent vs not detected vs unknown	Use explicit evidence states and never infer absence from failed extraction.	Blurry/cropped labels produce unknown/incomplete status, not a safe/absent claim.
Simple interface with image, extraction, preferences, flags, uncertainty	Responsive scan and report UI with separate panels/cards.	The user can see all required elements in one report.

Baseline requirements are the non-negotiable core. Consumption tracking, pattern alerts, and future wearable integrations extend the baseline; they do not replace it.

6. System architecture

7. Decision logic and status model

Status	Use when	Example wording
CONFLICT DETECTED	Readable evidence explicitly supports a conflict with a strict restriction.	“Peanut pieces” is listed in the ingredient text.
PREFERENCE MISMATCH	A value is available and exceeds a user-configured preference/target.	This serving would put your projected recorded sodium above your selected target.
NO CONFLICT IDENTIFIED IN READABLE INFORMATION	Relevant information is sufficiently readable and no matching ingredient was identified.	No peanut ingredient was identified in the readable ingredient list; this does not guarantee peanut-free status.
UNKNOWN / VERIFY	Information is missing, unreadable, ambiguous, or extraction is incomplete.	The ingredient list is partly obscured; peanut status cannot be verified.
NOT PROVIDED	The label does not provide the requested field.	Added sugar is not separately quantified on the visible label.

Do not collapse these states into one green/red score. A strict allergy conflict must remain prominent even if other preferences match. A self-reported LLM confidence percentage is not a calibrated accuracy guarantee; use observable evidence completeness/readability indicators instead.

8. Nutrition calculations and consumption ledger

Preserve the original label basis (per 100 g, per 100 ml, per serving, or per package). Ask the user for the amount they intend to consume, then calculate the nutrient contribution using a consistent unit and serving basis.

Nutrient consumed = label nutrient per 100 g/ml × amount consumed ÷ 100

Example: if a drink reports 10 g sugar per 100 ml and the user selects 250 ml, the estimated contribution is 25 g sugar. This is a calculation example, not a medical assessment.

- Analyze only: inspect a product without logging consumption.
- Preview impact: show the selected portion's contribution and projected total.
- Confirm consumption: add an event to the ledger only after confirmation.
- Daily view: calculate totals from events for the user's local calendar day.
- History: retain previous days for weekly summaries; do not delete history at midnight.
- Targets: reset the daily intake tally, not the user's configured target. Targets should not change automatically without a justified rule or user action.
- Avoid double-counting by allowing users to edit or remove an incorrectly logged event.

9. Feature scope: 24-hour MVP vs future roadmap

Feature	24-hour MVP	Later / stretch
Profile	Allergies, dietary preference, custom avoids, configurable sugar/sodium targets.	More profile options and account sync.
Label extraction	One authorized vision/OCR API; ingredients, nutrition, allergen text, source evidence.	More providers/languages and improved OCR fallback.
Evidence Lens	Source snippets tied to every finding.	Image bounding-box highlighting if OCR coordinates are available.
Uncertainty Radar	Missing/unreadable/ambiguous/incomplete states.	Guided retake prompts and section-level image-quality assistance.
Compatibility Map	Separate allergy, dietary, and nutrition results.	Product comparison and richer visualizations.
Nutrition Memory	Manual portion choice + confirmed logging + daily totals.	Meal grouping, barcode/product database, export.
Pattern alerts	Simple rule such as repeated logged energy-drink consumption.	User-configurable trends and scheduled summaries.
Wearables	Mock/demo interface only if clearly labeled; no live data required.	Real integration after choosing a supported device/platform and confirming permissions/data availability.
Medical recommendations	Do not diagnose or claim food will cause a condition; display label facts and user-defined targets.	Any clinical feature requires qualified review and validation.

10. Recommended stack and repository

Layer	Recommended option	Notes
Frontend	React OR HTML/CSS/JavaScript	Use the stack the team already knows; responsive web UI is enough.
Backend	Python + Flask	Upload endpoint, API calls, validation, matching, ledger endpoints.
AI	Authorized multimodal vision/OCR API	Check current access, quotas, pricing and image support before the event.
Rules & calculations	Python	Deterministic matching and nutrient arithmetic.
Persistence	SQLite for fastest prototype; Supabase if already familiar	Store profile, products, consumption events, findings.
Charts	Chart.js or simple HTML/CSS bars	Daily intake and history.
Version control / hosting	GitHub + compatible web host	Keep API secrets in environment variables.

```
LabelLens/\n■■■ app.py\n■■■ requirements.txt\n■■■ .env.example\n■■■ README.md\n■■■ backend/\n■ ■■■ extraction.py\n■ ■■■ normalization.py\n■ ■■■ evidence.py\n■ ■■■ uncertainty.py\n■ ■■■ matching.py\n■ ■■■ ledger.py\n■ ■■■ report.py\n■■■ frontend/\n■ ■■■ index.html (or React app)\n■ ■■■ style.css\n■ ■■■ script.js\n■■■ tests/\n ■■■ test_matching.py\n ■■■ test_uncertainty.py
```

- Never expose the AI API key in browser code or commit it to GitHub.
- Set upload size/type limits and handle provider errors/timeouts.
- Use mock JSON on the frontend while backend work continues.
- Define request/response JSON contracts in the first hour.

11. Six-person team division

Role	Primary ownership
1. Frontend lead / integrator	API contract, page structure, integration, coordination.
2. UI/UX designer	Profile, scan flow, report layout, responsive states.
3. Frontend developer	Upload, preferences, evidence cards, portion preview and dashboard.
4. AI/OCR developer	Vision API integration and structured extraction.
5. Backend developer	Flask endpoints, database schema, persistence.
6. Rules / QA developer	Matching rules, uncertainty states, nutrient math, test cases, deployment support.

Work in parallel using a shared API contract. The frontend can use mock responses immediately. Integrate frequently; do not wait until the final hours.

12. 24-hour execution plan

Time	Goal	Definition of done
0–2 h	Scope, repository, API test, JSON contract	One real image API call works; team responsibilities agreed.
2–6 h	Upload + structured extraction	User can upload a label and view extracted fields.
6–10 h	Validation, evidence and matching	Findings have source text; missing/unreadable states work.
10–13 h	Portion preview + consumption ledger	A confirmed event updates nutrient totals correctly.
13–16 h	Report UI + daily dashboard	Profile, image, extraction, flags, uncertainty and totals visible.
16–18 h	Basic pattern detection	Repeated logged pattern can be demonstrated with seeded/sample history.
18–20 h	Integration and edge-case tests	Clear, blurry, cropped, missing-value and API-error cases handled.
20–22 h	Deployment	Public demo works and secrets are configured safely.
22–24 h	README, demo video, final bug fixes	A rehearsed end-to-end demonstration and submission links are ready.

If behind schedule, cut wearable integration, accounts, product databases and advanced pattern analysis first. Do not cut source evidence, uncertainty handling, or the basic end-to-end scan.

13. Acceptance tests and demo script

Test	Expected result
Clear label with listed allergen	Conflict detected; exact source text shown.
Complete readable list with no matching ingredient	No match identified in readable list; no guarantee of absence.
Blurry/cropped ingredient list	Unknown/incomplete; user prompted to retake.
Nutrition value absent	Field is unavailable/unknown, not invented.
Nutrition value per 100 ml; portion 250 ml	Correct quantity conversion.
Scan product but do not consume	Daily ledger unchanged.
Confirm consumption	One timestamped event is recorded and totals update.
Repeated logged energy-drink pattern	Neutral pattern reminder based on recorded events.
Malformed API response / timeout	Graceful error and retry; no fabricated report.

0:00–0:20 — Problem

Explain label complexity and personalized dietary needs.

0:20–0:40 — Profile

Select a restriction and a user-defined nutrition target.

0:40–1:10 — Scan

Upload a real label; show extracted ingredients, nutrition, and source text.

1:10–1:35 — Evidence

Open a flag and show the exact label evidence.

1:35–1:55 — Uncertainty

Show a cropped/blurry label returning unknown rather than absent.

1:55–2:20 — Nutrition Memory

Preview a portion, confirm consumption, and show daily totals update.

2:20–2:40 — Patterns and close

Show a simple repeated-consumption insight and summarize differentiators.

14. Safety, privacy and honest limitations

- Do not claim a product is allergen-free just because an allergen was not detected. Precautionary statements and cross-contact may not be fully represented.
- Distinguish “contains” declarations from “may contain” precautionary statements.
- Do not invent missing ingredients, nutrition values, caffeine amounts, or source details.
- Keep allergy conflicts separate from optional nutrition preferences; never hide a strict restriction behind an overall score.
- Health conditions, weight, and wearable data are sensitive. Collect only what is needed, explain why, provide user control, and avoid storing unnecessary details.
- Wearable readings are device-dependent and may not be available or clinically reliable for every user. Clearly label simulated demo data.
- Do not diagnose disease or claim a particular product will cause harm. Describe label evidence, calculations, user-defined targets, and uncertainty.
- A target being exceeded is a comparison against that target, not automatically a medical emergency or proof of harm.

15. Final deliverables and success criteria

- GitHub repository with setup instructions, architecture, environment-variable example, screenshots, and limitations.
- Deployed demo or a rehearsed local fallback if hosting fails.
- Video presentation demonstrating the end-to-end flow, including evidence and an unknown case.
- At least one clear label, one allergen conflict, one unreadable/incomplete label, and one confirmed consumption event tested.
- A concise explanation of what is implemented now versus what is planned for later.

Definition of success: A real label image becomes structured, verifiable information; user preferences produce evidence-backed findings; uncertain information remains unknown; and confirmed consumption updates the user's daily ledger correctly.